CRIANDO PROJETOS EM PYTHON

Introdução

Quando iniciamos os estudos em lógica de programação, é comum escrevermos scripts únicos e lineares, onde todo o código reside em um único arquivo. No entanto, ao transicionarmos para o desenvolvimento de software profissional, essa abordagem se torna insustentável. Sistemas reais são complexos, exigem manutenção constante e, frequentemente, o trabalho de múltiplas pessoas. Nesse contexto, criar um projeto em Python vai muito além de salvar um arquivo com a extensão .py. Envolve isolar dependências, estruturar pastas de forma lógica e aplicar princípios de Orientação a Objetos para que o código seja modular, reutilizável e fácil de manter.

Um projeto Python organizado é aquele que separa responsabilidades de forma clara. Ele utiliza ambientes virtuais para gerenciar bibliotecas sem conflitar com outras aplicações no mesmo computador. Além disso, possui uma estrutura de diretórios padronizada, como uma pasta dedicada ao código fonte e outra para configurações, e divide a lógica de negócios em classes e módulos distintos. Essa organização previne o chamado "código espaguete", uma situação caótica onde variáveis, funções e regras de negócio estão todas misturadas, tornando a leitura e a depuração extremamente difíceis.

Dominar essa estrutura inicial é o divisor de águas entre um estudante que cria scripts soltos e um profissional capaz de desenvolver sistemas robustos. A organização estabelecida no primeiro dia de um projeto economiza horas de trabalho futuro, facilita a colaboração em equipe e estabelece uma base sólida para a adoção de ferramentas profissionais, como controle de versão e testes automatizados. Portanto, aprender a criar um projeto corretamente é o primeiro passo essencial para a engenharia de software.

Estruturando o Projeto na Prática

O Ambiente Virtual (venv)

O primeiro passo em qualquer projeto Python é a criação de um ambiente virtual. Imagine o ambiente virtual como uma caixa isolada onde seu projeto vive. Ele garante que as bibliotecas instaladas para este projeto específico não interfiram em outros projetos do seu computador, nem sofram interferência deles. Para criar um ambiente virtual nativo, utilizamos o comando `python -m venv venv` no terminal. Isso gera uma pasta chamada `venv` que contém uma cópia isolada do interpretador Python e do gerenciador de pacotes. Antes de programar, devemos ativá-lo: no Linux ou macOS, usa-se `source venv/bin/activate`; no Windows, utiliza-se `venv\Scripts\activate`.

A Estrutura de Pastas

Uma estrutura profissional mínima separa o código executável dos arquivos de configuração e dependências. Um padrão amplamente adotado na comunidade Python é:

- `venv/`: Pasta do ambiente virtual (nunca deve ser enviada para repositórios de código).
- `src/`: Pasta que contém o código fonte real do projeto, organizado em módulos e classes.
- `main.py`: Arquivo principal que orquestra a execução do programa.
- `requirements.txt`: Arquivo de texto que lista todas as dependências externas do projeto.

Aplicando Orientação a Objetos no Projeto

A modularidade é um pilar da Orientação a Objetos. Vamos criar um exemplo prático: um gerenciador simples de tarefas. Em vez de colocar toda a lógica no `main.py`,

criamos um arquivo `tarefa.py` dentro da pasta `src`. Isso aplica o Princípio da Responsabilidade Única, onde cada arquivo tem um propósito bem definido.

```
# Arquivo: src/tarefa.py
class Tarefa:
    def __init__(self, titulo: str, descricao: str):
        self.titulo = titulo          # Atributo que armazena o nome da tarefa
        self.descricao = descricao    # Atributo que armazena os detalhes da tarefa
        self.concluida = False        # Atributo de estado, inicia sempre como False

    def marcar_concluida(self):
        # Método que altera o estado interno do objeto para verdadeiro
        self.concluida = True

    def exibir_detalhes(self):
        # Método que retorna uma string formatada com base no estado atual
        status = "Concluída" if self.concluida else "Pendente"
        return f"Tarefa: {self.titulo} | Status: {status}"
```

Agora, no arquivo principal, importamos essa classe e a utilizamos. Isso demonstra o conceito de reutilização e separação de preocupações.

```
# Arquivo: main.py
from src.tarefa import Tarefa  # Importa a classe Tarefa do módulo na pasta src

def main():
    # Cria uma instância (objeto) da classe Tarefa com dados iniciais
    minha_tarefa = Tarefa("Estudar Python", "Revisar criação de projetos")

    # Chama o método para exibir o estado inicial do objeto recém-criado
    print(minha_tarefa.exibir_detalhes())

    # Altera o estado do objeto chamando seu método específico de modificação
    minha_tarefa.marcar_concluida()

    # Verifica o novo estado, demonstrando o encapsulamento do comportamento
```

```
    print(minha_tarefa.exibir_detalhes())

if __name__ == "__main__":
    main() # Executa a função principal apenas se este arquivo for o ponto de entrada
```

O bloco `if __name__ == "__main__":` é uma convenção fundamental em Python. Ele verifica se o arquivo está sendo executado diretamente como programa principal ou se foi importado como um módulo por outro arquivo. Isso garante que o código dentro dele só seja executado quando desejamos iniciar o programa, evitando efeitos colaterais indesejados durante a importação de classes em outros contextos.

Conclusão

A criação de um projeto em Python é a fundação de qualquer desenvolvimento de software profissional. Ao adotar ambientes virtuais, garantimos a reprodutibilidade e o isolamento das dependências, evitando conflitos catastróficos entre bibliotecas. Ao estruturar pastas como `src` e arquivos como `main.py`, estabelecemos uma arquitetura clara que qualquer desenvolvedor consegue navegar e compreender rapidamente. Por fim, ao aplicar a Orientação a Objetos, dividindo responsabilidades em classes e módulos distintos, transformamos scripts frágeis e difíceis de manter em sistemas modulares, testáveis e escaláveis.

Dominar esses conceitos iniciais não apenas melhora a qualidade imediata do seu código, mas também prepara o terreno para práticas avançadas, como testes unitários, integração contínua e trabalho colaborativo em grandes equipes de desenvolvimento. O hábito de começar bem é o que diferencia um programador iniciante de um engenheiro de software em formação. Continue praticando a criação de novos projetos, sempre priorizando a organização e a clareza desde a primeira linha de código escrita.